

AUDITORÍA TÉCNICA DE ARQUITECTURA

PROYECTO EDARSAHUB

Fecha: 08/04/2026 12:17

A. MÓDULOS DETECTADOS

#	Módulo	Archivo	Líneas	Complejidad
1	Recursos Humanos	RecursosHumanos.js	3,413	ALTA
2	Reportes	Reportes.js	3,214	ALTA
3	Compras	Compras.js	2,887	ALTA
4	Comercial	Comercial.js	2,470	MEDIA-ALTA
5	Mis Tareas	MisTareas.js	1,780	MEDIA
6	Usuarios	Usuarios.js	1,438	MEDIA
7	Nóminas	Nominas.js	1,324	MEDIA
8	Explorador BD	ExploradorBD.js	1,241	MEDIA
9	Finanzas	Finanzas.js	1,090	MEDIA-BAJA
10	Servidores	Servidores.js	977	MEDIA-BAJA
11	Catálogo SQL	CatalogoConsultas.js	861	BAJA
12	Dashboard	Dashboard.js	748	BAJA
13	Autorización Compras	AutorizacionCompras.js	735	BAJA
14	Tablero Ejecutivo	TableroEjecutivo.js	695	BAJA
15	Portal Proveedores	Proveedores.js	495	BAJA
16	Alertas	Alertas.js	303	BAJA
17	Reportes BI	ReportesBI.js	58	MÍNIMA

Backend: 17,352 líneas en un solo archivo server.py (monolito)

B. STORES DETECTADOS

Tipo	Ubicación	Estado
Context Global	No existe	NO HAY
Redux/Zustand	No existe	NO HAY
IndexedDB (Local-First)	/services/localDB.js	EXISTE (poco usado)

Stores IndexedDB disponibles: kpis_tablero, inventarios, auditorias, catalogos, sync_log, prefetch_cache

Hooks disponibles: useLocalFirst.js (caché local), useToast.js (notificaciones)

C. FILTROS DETECTADOS

Módulo	Server	Sucursal	Período	Otros	Storage
Comercial	LOCAL	LOCAL	LOCAL	Tab	comercial_filters
Compras	LOCAL	LOCAL	LOCAL	Tab, Inv.	compras_params
Reportes	LOCAL	LOCAL	LOCAL	Tipo	reportes_filters
Dashboard	LOCAL	N/A	N/A	-	-
Tablero Ejecutivo	N/A	N/A	LOCAL	Mes, Año	tablero_filtros
Explorador BD	LOCAL	N/A	N/A	Tablas	-
Catálogo SQL	LOCAL	N/A	N/A	-	-

Nota: Cada módulo tiene su propio useState para: servers, selectedServer, selectedSucursal, sucursales

D. PROBLEMAS DE ARQUITECTURA DETECTADOS

CRÍTICOS

#	Problema	Ubicación	Impacto
1	Monolito Backend (17,352 líneas)	server.py	Difícil mantener
2	Código duplicado de filtros	Cada módulo	Inconsistencias
3	Sin Context Global para Server	Todos los módulos	Fetch repetido
4	Componentes internos gigantes	Comercial, Compras, Reportes	Difícil mantener

MEDIOS

#	Problema	Ubicación	Impacto
5	LocalStorage como comunicación	comercial_filters, etc.	Desincronización
6	Hook useLocalFirst poco usado	Hooks	Optimización perdida
7	Sin validación de tipos	Frontend	JavaScript vanilla
8	Filtros repetidos en cada Tab	Comercial, Compras	UX inconsistente

E. DEPENDENCIAS CRUZADAS DETECTADAS

Recurso Compartido	Módulos que lo usan
/api/servers	Comercial, Compras, Reportes, Dashboard, Explorador, Catálogo SQL
/api/servers/{id}/sucursales	Comercial, Compras, Reportes
visible_en_operaciones filter	Comercial, Compras, Reportes, Dashboard, Tablero Ejecutivo
localStorage.token	TODOS los módulos

F. RIESGOS CRÍTICOS

#	Riesgo	Prob.	Impacto	Módulos Afectados
1	Cambiar /api/servers	ALTA	ALTO	Comercial, Compras, Reportes, Dashboard...
2	Cambiar visible_en_operaciones	MEDIA	ALTO	Todos los módulos operativos
3	Modificar estructura sucursales	MEDIA	MEDIO	Comercial, Compras, Reportes
4	Cambiar execute_sql_query()	BAJA	ALTO	TODOS los queries SQL
5	Modificar modelos Pydantic	MEDIA	MEDIO	Endpoints relacionados

G. ELEMENTOS ESTABLES QUE CONVIENE CONGELAR

NO TOCAR (Funcionan bien y son compartidos):

Elemento	Razón
/api/servers endpoint	Base de todos los módulos
/api/servers/{id}/sucursales endpoint	Crítico para filtros
execute_sql_query() función	Motor de todas las consultas SQL
get_current_user() función	Autenticación central
visible_en_operaciones filtro	Ya implementado en todos los módulos
Componentes UI en /components/ui/	Shadcn genéricos sin lógica de negocio
Sistema de caché offline (kpis_cache)	Funciona correctamente
Flujo de autenticación	Estable

RESUMEN EJECUTIVO

Categoría	Estado	Detalle
Arquitectura Frontend	MEDIO	Modular pero con código duplicado
Arquitectura Backend	CRÍTICO	Monolito - riesgo alto
Aislamiento de Módulos	BUENO	Cada módulo es independiente
Filtros Globales	MEDIO	Ninguno centralizado
Dependencias Cruzadas	MEDIO	Solo vía endpoints compartidos
Riesgo de Cambios	MEDIO	Endpoints compartidos son el riesgo

--- Fin del Documento ---